

THỰC HÀNH VI XỬ LÝ – VI ĐIỀU KHIỂN

GVHD: Trần Ngọc Đức

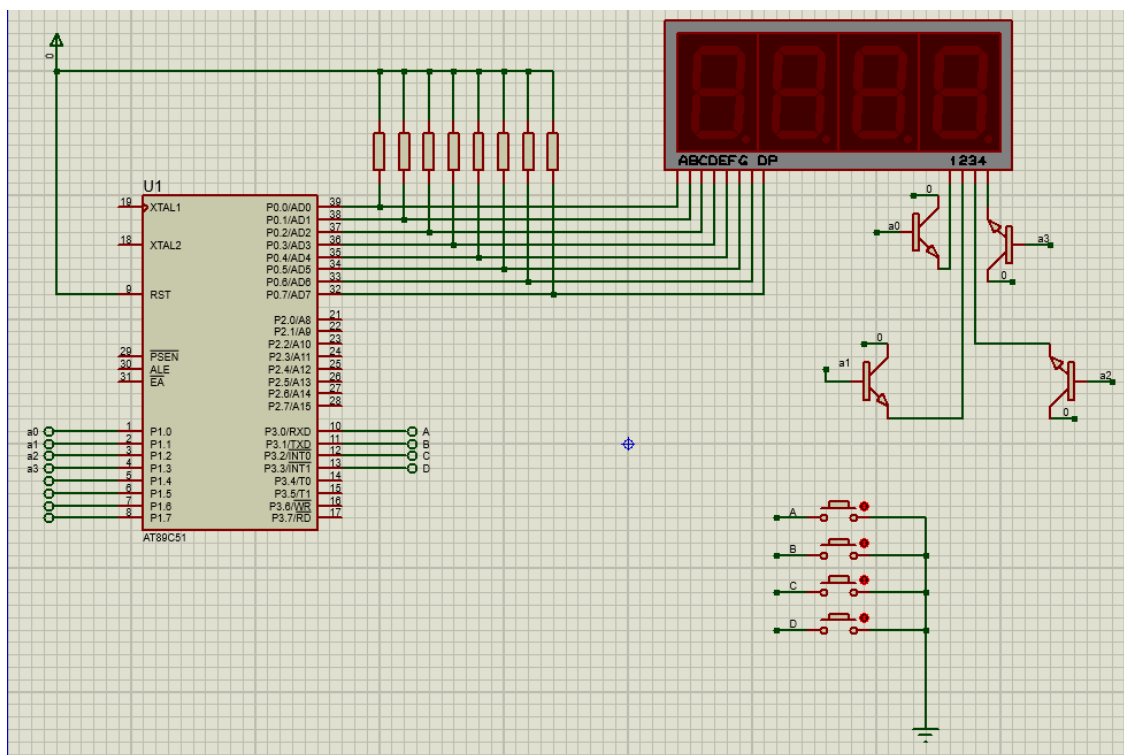
Họ và tên sinh viên thực hiện: Vũ Đại Dương

Mã số sinh viên: 23520359

BÀI THỰC HÀNH 05: SỬ DỤNG INTERRUPT

I. Mô phỏng trên protues

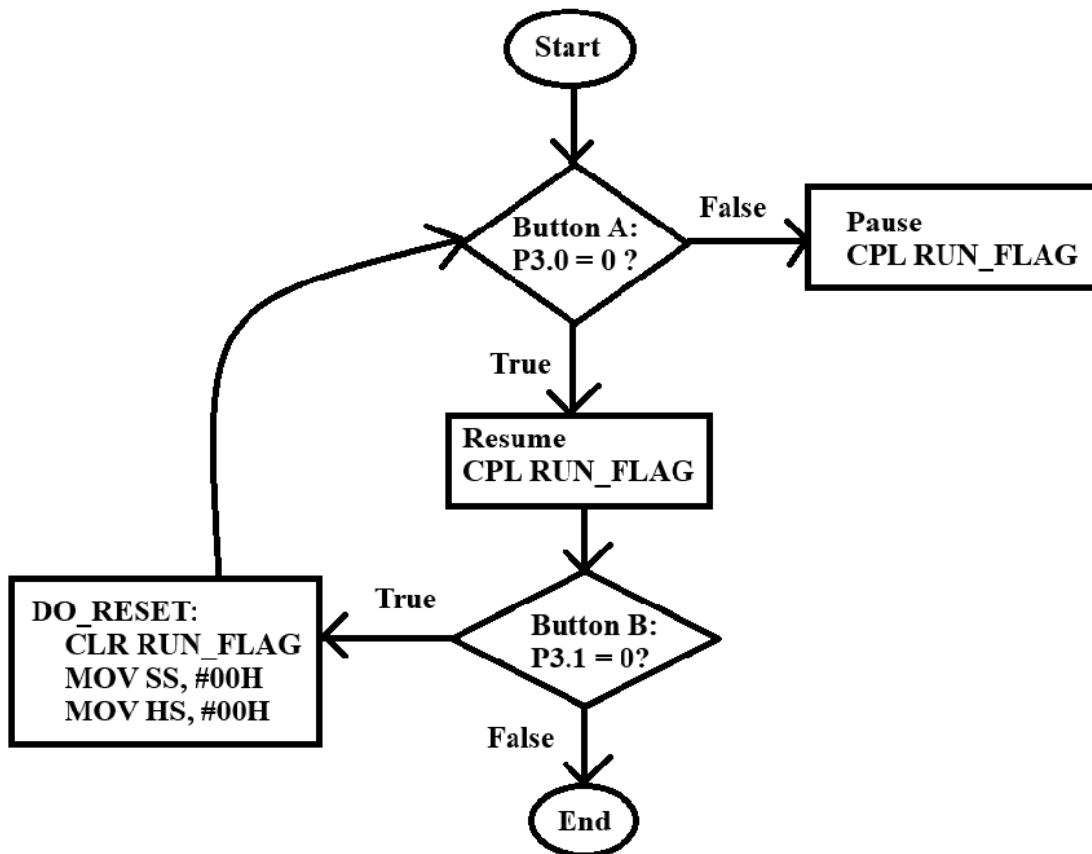
Hình ảnh mô phỏng trên protues:



- Do thời gian trên protues với thời gian thực tế có một chút chênh lệch, nên đồng hồ đếm sẽ cài đặt thời gian đếm bằng với thời gian protues.
- Link drive kết quả chạy đồng hồ:

https://drive.google.com/file/d/1_dgN98oWhQoFD4Kxnr9wnmteeyCQwdNQ/view?usp=sharing

- II. Trình bày và vẽ lưu đồ giải thuật xử lý 2 button có chức năng sau
- o Button A: Pause/Resume đồng hồ bấm giờ
 - o Button B: Reset đồng hồ.



III. Giải thích code

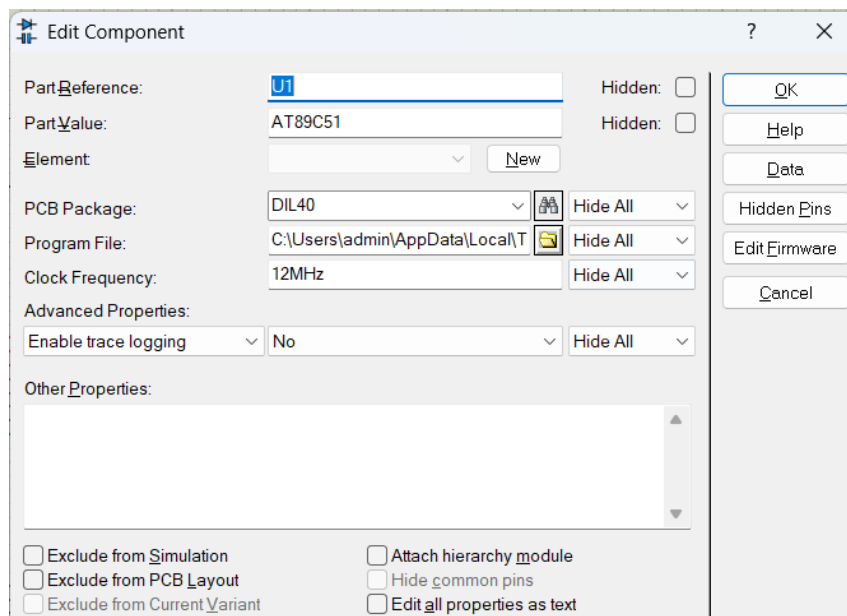
Source code	Giải thích
<pre> ORG 0000H LJMP MAIN ORG 000BH TIMER0_ISR: MOV TH0, #0D8H MOV TL0, #0F0H JNB RUN_FLAG, SKIP_UPDATE MOV A, HS INC A MOV HS, A CJNE A, #100, SKIP_UPDATE MOV HS, #00H ACALL INCREMENT_SECOND SKIP_UPDATE: RETI MAIN: MOV TMOD, #01H MOV TH0, #0D8H MOV TL0, #0F0H SETB TR0 MOV IE, #82H CLR RUN_FLAG MOV HS, #00H MOV SS, #00H MAIN_LOOP: ACALL REFRESH_DISPLAY ACALL CHECK_BUTTONS SJMP MAIN_LOOP CHECK_BUTTONS: JB P3.0, SKIP_BTN_A ACALL DEBOUNCE JNB P3.0, TOGGLE_RUN </pre>	Đặt chương trình bắt đầu tại địa chỉ 0x0000 Nhảy với hàm Main. Vecto ngắt của timer 0. Khi timer 0 tràn, chương trình sẽ nhảy tới TIMER0_ISR. Nạp giá trị cho timer 0 tương đương 1ms. Đây là một cờ RUN_FLAG nếu đồng hồ đang ngừng thì không tăng thời gian. HS là biến đếm chu kỳ để đạt 1 giây. Khi HS đạt 100 tương đương 1 giây vì mỗi ngắt chỉ 1ms. Gọi hàm tăng giây. Trả về từ interrupt. Cấu hình timer 0 ở chế độ 1. Nạp giá trị timer. Khởi động timer 0. Bật ngắt toàn cục và ngắt timer 0. Khởi tạo các biến về 0. RUN_FLAG xác định trạng thái resume/pause. Gọi hiển thị LED và kiểm tra nút bấm trong vòng lặp vô hạn. Kiểm tra các nút tại A, B, C, D tương đương P3.0 đến P3.3. Nếu chưa nhấn nút A, thì bỏ qua đến các nút còn lại. Nếu nhấn thì đến hàm TOGGLE_RUN là hàm resume/pause đồng hồ.

SKIP_BTN_A: JB P3.1, SKIP_BTN_B ACALL DEBOUNCE JNB P3.1, DO_RESET SKIP_BTN_B: JB P3.2, SKIP_BTN_C ACALL DEBOUNCE JNB P3.2, DO_INC_SEC SKIP_BTN_C: JB P3.3, SKIP_BTN_D ACALL DEBOUNCE JNB P3.3, DO_DEC_SEC SKIP_BTN_D: RET TOGGLE_RUN: CPL RUN_FLAG WAIT_REL_A: JNB P3.0, WAIT_REL_A RET DO_RESET: CLR RUN_FLAG MOV SS, #00H MOV HS, #00H WAIT_REL_B: JNB P3.1, WAIT_REL_B RET DO_INC_SEC: MOV A, SS INC A CJNE A, #60, SKIP_WRAP_INC CLR A SKIP_WRAP_INC: MOV SS, A WAIT_REL_C: JNB P3.2, WAIT_REL_C RET DO_DEC_SEC: MOV A, SS	Nếu nhấn nút B thì đến hàm DO_RESET là reset giá trị đồng hồ. Nếu nhấn nút C thì đến hàm DO_INC_SEC tăng 1 giây. Nếu nhấn nút D thì đến hàm DO_DEC_SEC giảm 1 giây. Nếu không nhấn nút nào sẽ quay về. Đảo bit RUN_FLAG: nếu đồng hồ đang chạy thì dừng và ngược lại. Kiểm tra nút còn chưa bật, tức là đang giữ nút thì đợi đến khi nút bật. Dừng đồng hồ. Đặt giây về 0. Đặt tích tắc về 0. Kiểm tra nút còn chưa bật, tức là đang giữ nút thì đợi đến khi nút bật. Tăng giây. Nếu chưa đến 60s thì nhảy đến SKIP_WRAP_INC. A = 0. Ghi lại giá trị vào biến SS. Kiểm tra nút còn chưa bật, tức là đang giữ nút thì đợi đến khi nút bật. Chuyển giá trị biến SS cho A.
--	--

<pre> JZ NO_DEC DEC A NO_DEC: MOV SS, A WAIT_REL_D: JNB P3.3, WAIT_REL_D RET INCREMENT_SECOND: MOV A, SS INC A CJNE A, #60, NO_WRAP CLR A NO_WRAP: MOV SS, A RET REFRESH_DISPLAY: MOV A, SS MOV B, #10 DIV AB MOV DPTR, #SEG7 MOVC A, @A+DPTR MOV P0, A MOV P1, #01H ACALL SMALL_DELAY MOV P1, #00H MOV A, B MOV DPTR, #SEG7 MOVC A, @A+DPTR ANL A, #7FH MOV P0, A MOV P1, #02H ACALL SMALL_DELAY MOV P1, #00H MOV A, HS MOV B, #10 DIV AB MOV DPTR, #SEG7 MOVC A, @A+DPTR MOV P0, A </pre>	Nếu A = 0; thì không giảm. Giảm giây. Ghi giá trị mới cho biến SS. Kiểm tra nút còn chưa bật, tức là đang giữ nút thì đợi đến khi nút bật. Hàm tăng giây. Tăng giây mỗi khi HS đạt 100. Nếu chưa bằng 60 thì nhảy đến NO_WRAP. A = 0. Ghi giá trị mới từ A cho biến SS. Hàm hiển thị LED. Hiển thị LED giây. Chia để tách số hàng chục và hàng đơn vị. Truy xuất mã LED 7 đoạn từ bảng SEG7. A = SS/10 (hàng chục), B = SS%10 (hàng đơn vị). Lấy mã LED từ bảng SEG7 các số từ 1 đến 9. Chuyển vào P0 để hiển thị số. Chọn LED đầu tiên. Gọi hàm small_delay để mắt người có thể nhìn thấy. Tắt LED để quét LED khác. Hiển thị LED hàng đơn vị. Lấy mã LED từ bảng SEG7 các số từ 1 đến 9. Hiển thị dấu “ . ” Chuyển vào P0 để hiển thị số. Chọn LED thứ hai. Tắt LED. Hiển thị LED tích tắc. A = SS/10 (hàng chục), B = SS%10 (hàng đơn vị). Hiển thị LED hàng chục.
--	--

<pre> MOV P1, #04H ACALL SMALL_DELAY MOV P1, #00H MOV A, B MOV DPTR, #SEG7 MOVC A, @A+DPTR MOV P0, A MOV P1, #08H ACALL SMALL_DELAY MOV P1, #00H RET SMALL_DELAY: MOV R2, #100 DL_LOOP: DJNZ R2, DL_LOOP RET DEBOUNCE: MOV R3, #255 DB_LOOP: DJNZ R3, DB_LOOP RET DSEG HS: DS 1 SS: DS 1 RUN_FLAG BIT 20H CSEG SEG7: DB 0C0H,0F9H,0A4H, 0B0H,099H,092H,082H,0F8H,080H,090H END </pre>	Chọn LED thứ 3. Tắt LED. Hiển thị LED hàng đơn vị. Chọn LED thứ 4. Tắt LED. Hàm Delay để mắt người có thể nhìn thấy. Tương tự như small_delay nhưng dài hơn. Dùng sau khi phát hiện nhấn nút để tránh nhiễu. HS và SS là 2 thanh ghi dạng byte lưu thời gian. Byte đếm tích tắc. Byte đếm giây. Bit cờ xác định trạng thái đồng hồ. Bảng mã LED. Từ 0 -> 9.
--	---

Giải thích cách tính timer 0:



Cài đặt thạch anh cho AT89C51 là 12 Mhz.

$$F = 12\text{M} / 12 = 1\text{M}.$$

$$T = 1 / 1\text{M}.$$

Tần số xung cần cài đặt là 0.01s (tức là 1 tích tắc).

$$N = 0.01\text{s} / (1 / 1\text{M}) = 10000.$$

$$\text{Giá trị cần cài đặt } 65536 - 10000 = 55536_{\text{DEC}} = \text{D8F0}_{\text{HEX}}..$$

$$\text{TH0} = \text{D8}.$$

$$\text{TL0} = \text{F0}.$$

Giải thích cách bật LED

LED muốn bật	Nhi phân	Thập lục phân
1	0000 0001	0x01
2	0000 0010	0x02
3	0000 0100	0x04
4	0000 1000	0x08